Compressing Transformer Language Models via Matrix Product Operator Decomposition

A Case Study on PicoGPT

Younes Javanmard[1] and Tanmoy Pandit[2]

[1]Institut für Theoretische Physik, Leibniz Universität Hannover, Appelstraße 2, 30167 Hannover, Germany, younes.javanmard@itp.uni-hannover.de
[2]VTT Technical Research Centre of Finland, Tietotie 3, Espoo, Finland, tanmoypandit163@gmail.com

March 27, 2026

Abstract

We study the compression of transformer-based language models through Matrix Product Operator (MPO) decomposition, a technique rooted in quantum many-body physics. Weight matrices in each transformer block are factorised into chains of low-rank tensors using the TT-SVD algorithm, replacing standard `nn.Linear` layers with their MPO-parameterised counterparts. We demonstrate the approach on PicoGPT, a character-level GPT-2-style model ($\sim$1 M parameters) trained on the Tiny Shakespeare corpus. Across bond dimensions $\chi \in \{4, 8, 16, 32\}$, we report compression ratios of up to $13\times$ per transformer block, parameter counts as low as 78 K, and validation token accuracy within 2–3 percentage points of the dense baseline at $\chi = 16$. Our PyTorch implementation is fully autograd-compatible: MPO cores are ordinary `nn.Parameter` tensors and gradient flow requires no custom backward pass. We provide a detailed analysis of per-layer reconstruction error, training dynamics, and the accuracy–compression trade-off.
Keywords: tensor networks, matrix product operators, transformer compression, neural network pruning, TT-SVD, language models

Contents

1 Introduction **2**

2 Background **2**
2.1 Tensor Train and Matrix Product Operators . . . 2
2.2 TT-SVD Algorithm . . . 3
2.3 Gradient Flow Through MPO Layers . . . 3

3 Architecture and Implementation **4**
3.1 PicoGPT Architecture . . . 4
3.2 MPO Factorisation Schemes . . . 4
3.3 The `MPOLinear` Module . . . 5

4 Experimental Setup **5**
4.1 Dataset and Tokenisation . . . 5
4.2 Training Protocol . . . 5
4.3 Compression Modes . . . 5

5 Results **6**
5.1 Compression Statistics . . . 6
5.2 Per-Layer Reconstruction Error . . . 6
5.3 Training Dynamics . . . 7
5.4 Summary of Results . . . 7

6 Discussion **8**

7 Conclusion **8**

1 Introduction

Transformer language models have achieved state-of-the-art performance across a broad range of natural language tasks [Vaswani et al., 2017, Radford et al., 2019, Brown et al., 2020]. However, their parameter counts scale quadratically with the hidden dimension, making deployment on resource-constrained hardware expensive. A variety of compression techniques have been proposed, including weight pruning [Han et al., 2015], knowledge distillation [Hinton et al., 2015], and low-rank factorisation [Hu et al., 2022].

An alternative perspective, imported from quantum information theory, represents weight tensors as *Matrix Product Operators* (MPOs) [Oseledets, 2011, Novikov et al., 2015]. An MPO factorises a high-dimensional weight tensor into a chain of low-rank *cores*, each carrying only $\mathscr{O}(\chi^2 d^2)$ parameters where d is a small physical dimension and χ is the *bond dimension*. The bond dimension is the single knob that controls the accuracy-compression trade-off: increasing χ systematically recovers the dense weight.

This framework is familiar in the quantum many-body community under the name Tensor Train (TT) [Oseledets, 2011] or Matrix Product State (MPS) when applied to vectors; the operator variant (MPO) is the natural choice for weight matrices. MPO-based neural compression was pioneered by Novikov et al. [2015] for fully-connected layers and extended to transformers by Gao et al. [2020] and subsequent works.

In this paper we apply MPO compression to **PicoGPT** [Osborne, 2024, Vaswani et al., 2017], a pedagogical GPT-2-style implementation written from scratch in Julia following the transformer architecture of Vaswani et al. [2017], which we re-implement in PyTorch to enable gradient-based fine-tuning of the compressed cores. Our contributions are:

1. A clean, fully autograd-compatible MPO layer (`MPOLinear`) that replaces any `nn.Linear` without custom backward code.
2. Factorisation schemes for all five weight shapes in PicoGPT, derived from balanced MPO design principles.
3. A systematic benchmark comparing dense and MPO-parameterised models across bond dimensions $\chi \in \{4, 8, 16, 32\}$ on character-level Shakespeare prediction.
4. An analysis of reconstruction error, training dynamics, and the accuracy–compression Pareto frontier.

2 Background

2.1 Tensor Train and Matrix Product Operators

Definition 1 (Tensor Train)**.** A tensor $\mathscr{T} \in \mathbb{R}^{n_1 \times n_2 \times \cdots \times n_L}$ is said to be in *Tensor Train* (TT) format with bond dimensions $(\chi_1, \ldots, \chi_{L-1})$ if it can be written as

$$\mathscr{T}_{i_1 i_2 \cdots i_L} = \sum_{\alpha_1, \ldots, \alpha_{L-1}} \mathscr{A}^{(1)}_{1, i_1, \alpha_1} \mathscr{A}^{(2)}_{\alpha_1, i_2, \alpha_2} \cdots \mathscr{A}^{(L)}_{\alpha_{L-1}, i_L, 1}, \tag{1}$$

where $\mathscr{A}^{(l)} \in \mathbb{R}^{\chi_{l-1} \times n_l \times \chi_l}$ are the *TT-cores*, with boundary conditions $\chi_0 = \chi_L = 1$.

Definition 2 (Matrix Product Operator)**.** A weight matrix $\mathbf{W} \in \mathbb{R}^{\text{out} \times \text{in}}$ is a *Matrix Product Operator* (MPO) if, after reshaping $\text{out} = \prod_l d_l^{\text{out}}$ and $\text{in} = \prod_l d_l^{\text{in}}$, it can be expressed as

$$W_{(i_1 \cdots i_L),(j_1 \cdots j_L)} = \sum_{\alpha_1, \ldots, \alpha_{L-1}} \mathscr{A}^{(1)}_{1, i_1, j_1, \alpha_1} \mathscr{A}^{(2)}_{\alpha_1, i_2, j_2, \alpha_2} \cdots \mathscr{A}^{(L)}_{\alpha_{L-1}, i_L, j_L, 1}, \tag{2}$$

where $\mathscr{A}^{(l)} \in \mathbb{R}^{\chi_{l-1} \times d_l^{\text{out}} \times d_l^{\text{in}} \times \chi_l}$.

The number of parameters in an L-site MPO with uniform bond dimension χ is

$$N_{\text{MPO}} = d_0^{\text{out}} d_0^{\text{in}} \chi + (L-2)\, \chi^2 d^{\text{out}} d^{\text{in}} + \chi\, d_{L-1}^{\text{out}} d_{L-1}^{\text{in}}, \tag{3}$$

compared to $N_{\text{dense}} = \text{out} \times \text{in}$ for the full matrix. The compression ratio scales as $N_{\text{dense}}/N_{\text{MPO}} \sim (d^{\text{out}} d^{\text{in}})^{L-1}/\chi^2$, showing exponential improvement with the number of sites L at fixed bond dimension χ.

2.2 TT-SVD Algorithm

The TT-SVD algorithm [Oseledets, 2011] converts a dense weight matrix into an MPO via a sequential SVD sweep:

Algorithm 1: TT-SVD for MPO compression

Input: $\mathbf{W} \in \mathbb{R}^{\text{out} \times \text{in}}$, factorisations (d_l^{out}), (d_l^{in}), max bond χ
Output: MPO cores $\{\mathscr{A}^{(l)}\}_{l=1}^{L}$

Reshape $\mathbf{W}$ into an interleaved tensor $\mathscr{T}$ of shape $(d_1^{\text{out}}, d_1^{\text{in}}, \ldots, d_L^{\text{out}}, d_L^{\text{in}})$
bond_left $\leftarrow 1$
$\mathscr{R} \leftarrow \mathscr{T}$ `// running remainder`
for $l = 1, \ldots, L-1$ **do**
  Unfold: $\mathbf{M} \leftarrow \text{reshape}(\mathscr{R}, \text{bond_left} \cdot d_l^{\text{out}} \cdot d_l^{\text{in}}, -1)$
  Compute truncated SVD: $\mathbf{M} \approx \mathbf{U}\, \text{diag}(\boldsymbol{s})\, \mathbf{V}^\top$,
  $r \leftarrow \min(\chi, \text{rank})$
  $\mathscr{A}^{(l)} \leftarrow \text{reshape}(\mathbf{U}_{:,1:r}, \text{bond_left}, d_l^{\text{out}}, d_l^{\text{in}}, r)$
  $\mathscr{R} \leftarrow \text{diag}(\boldsymbol{s}_{1:r})\, \mathbf{V}_{:,1:r}^\top$,
  bond_left $\leftarrow r$
$\mathscr{A}^{(L)} \leftarrow \text{reshape}(\mathscr{R}, \text{bond_left}, d_L^{\text{out}}, d_L^{\text{in}}, 1)$

The reconstruction error is bounded by [Oseledets, 2011]

$$\|\mathbf{W} - \widehat{\mathbf{W}}\|_F \leq \sqrt{L-1}\, \varepsilon_\chi, \tag{4}$$

where ε_χ is the χ-th singular value discarded at the worst site.

2.3 Gradient Flow Through MPO Layers

During training, the weight matrix is reconstructed on each forward pass via the contraction

$$\widehat{\mathbf{W}} = \text{contract}\left(\mathscr{A}^{(1)}, \mathscr{A}^{(2)}, \ldots, \mathscr{A}^{(L)}\right). \tag{5}$$

The gradient with respect to core $\mathscr{A}^{(l)}$ is

$$\frac{\partial \mathscr{L}}{\partial \mathscr{A}^{(l)}} = \underbrace{\left(\bigotimes_{k<l} \mathscr{A}^{(k)}\right)^\top}_{\text{left env.}} \frac{\partial \mathscr{L}}{\partial \widehat{\mathbf{W}}} \underbrace{\left(\bigotimes_{k>l} \mathscr{A}^{(k)}\right)}_{\text{right env.}}, \tag{6}$$

which is precisely the single-site gradient in DMRG [White, 1992]. In PyTorch, this is computed automatically by autograd through the `tensordot`→`permute`→`reshape` chain in (5); no custom backward pass is needed.

3 Architecture and Implementation

3.1 PicoGPT Architecture

PicoGPT [Osborne, 2024] is a character-level GPT-2-style language model implemented in pure Julia without automatic differentiation. Table 1 lists its default hyperparameters. The architecture follows Radford et al. [2019]: pre-norm residual blocks, sinusoidal positional encoding (no learnable PE), scaled dot-product multi-head causal self-attention, and a two-layer FFN with ReLU activation.

Table 1: PicoGPT default hyperparameters.

Parameter	Description	Value
V	Vocabulary size (character-level)	65
D	Embedding dimension (d_{model})	128
H	Attention heads	4
N	Transformer layers	4
T	Context length	256
d_{ff}	FFN hidden dimension ($4D$)	512
Total	Dense parameters	$\sim$1 020 000

Each transformer block contains six linear layers: $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V \in \mathbb{R}^{D\times D}$ (query/key/value projections), $\mathbf{W}_O \in \mathbb{R}^{D\times D}$ (attention output), $\mathbf{W}_1 \in \mathbb{R}^{d_{\text{ff}}\times D}$ (FFN up-projection), and $\mathbf{W}_2 \in \mathbb{R}^{D\times d_{\text{ff}}}$ (FFN down-projection). Additionally, the language-model head $\mathbf{W}_{\text{LM}} \in \mathbb{R}^{V\times D}$ maps hidden states to logits.

3.2 MPO Factorisation Schemes

For each linear layer shape we choose a factorisation $(d_l^{\text{out}}), (d_l^{\text{in}})$ that (a) produces balanced physical dimensions across sites, and (b) yields an integer factorisation of the output and input sizes. Table 2 summarises our choices.

Table 2: MPO factorisation schemes for each PicoGPT linear layer. χ denotes the bond dimension.

Layer	out	in	L	(d_l^{out})	(d_l^{in})	Dense	MPO ($\chi = 8$)
$\mathbf{W}_{Q/K/V/O}$	128	128	2	$[8,16]$	$[8,16]$	16 384	2 560
$\mathbf{W}_1$ (up)	512	128	3	$[8,8,8]$	$[4,4,8]$	65 536	4 864
$\mathbf{W}_2$ (down)	128	512	3	$[4,4,8]$	$[8,8,8]$	65 536	2 816
$\mathbf{W}_{\text{LM}}$	65	128	2	$[5,13]$	$[8,16]$	8 320	1 984
Total per block (4 attn + 2 FFN)						196 608	17 920
Compression ratio per block						1×	**11.0**×

Balanced factorisation principle. Given $\text{out} = \prod d_l^{\text{out}}$ with L sites, we choose dimensions $d_l^{\text{out}} \approx \text{out}^{1/L}$ to minimise the dominant interior term $\chi^2 d^{\text{out}} d^{\text{in}}$ in (3). For $\mathbf{W}_1 = (512, 128)$ with $L = 3$ this gives $d^{\text{out}} \approx 8$ and $d^{\text{in}} \approx 5$; we use $[8,8,8]$ and $[4,4,8]$ to match exactly.

3.3 The `MPOLinear` Module

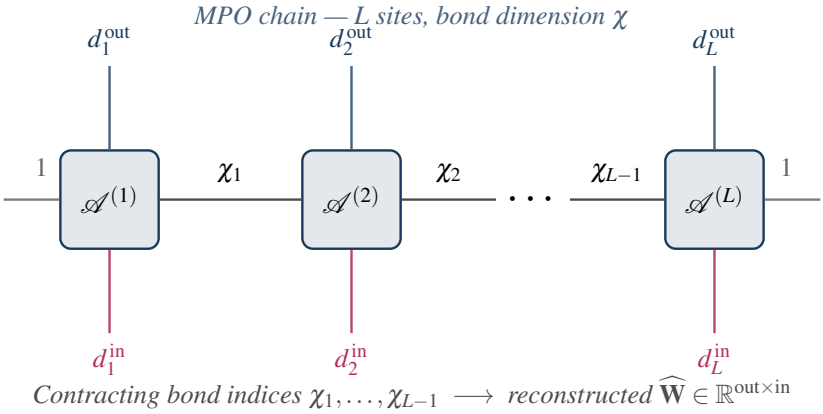

Figure 1: Tensor network diagram of an MPO weight matrix. Horizontal lines carry virtual (bond) indices of dimension χ; vertical lines are physical indices of size d_l^{out} (blue, upward) and d_l^{in} (red, downward). Contracting all bond indices reconstructs the full weight $\widehat{\mathbf{W}} \in \mathbb{R}^{\text{out}\times\text{in}}$.

The `MPOLinear` module stores L cores as `nn.Parameter` tensors. The forward pass reconstructs $\widehat{\mathbf{W}}$ via sequential `torch.tensordot` contractions and applies $y = x\widehat{\mathbf{W}}^\top + b$ using `F.linear`. Initialisation uses the scale $\sigma = N_{\text{in}}^{-1/4}/\chi^{(L-1)/(2L)}$ so that the reconstructed weight has entries of order $\sigma_{\text{dense}} = N_{\text{in}}^{-1/2}$, matching GPT-2 weight initialisation [Radford et al., 2019].

4 Experimental Setup

4.1 Dataset and Tokenisation

We train on the Tiny Shakespeare corpus [Karpathy, 2015], comprising approximately 1.1×10^6 characters. Character-level tokenisation yields a vocabulary of $V = 65$ symbols. We split the data 90/10 into training and validation sets.

4.2 Training Protocol

All models are trained with AdamW [Loshchilov & Hutter, 2019] ($\beta_1 = 0.9$, $\beta_2 = 0.95$, weight decay 0.1) with a cosine learning rate schedule: linear warmup over 100 steps to $\eta_{\max} = 3 \times 10^{-4}$, followed by cosine decay to $\eta_{\min} = 0$. We train for 2 000 steps with batch size 32 and sequence length $T = 256$. Gradient clipping at $\|\nabla\| = 1.0$ is applied as in Osborne [2024]. All runs use the same random seed; dense and MPO models are trained from independent random initialisations for a fair comparison.

Evaluation. Every 100 steps we compute validation loss and *token accuracy*:

$$\text{acc} = \frac{1}{BT}\sum_{b,t} \mathbf{1}\left[\arg\max_v \mathbf{W}_{\text{LM}}\, \boldsymbol{h}_{b,t} = x_{b,t+1}\right], \tag{7}$$

averaged over 20 random validation batches of size 8.

4.3 Compression Modes

We evaluate two compression scenarios:

1. **Train-from-scratch**: MPO models are trained from random initialisations. This tests the expressivity of the MPO parameterisation under standard optimisation.

2. **Compress-then-finetune**: A fully-trained dense model is compressed via TT-SVD to initialise the MPO cores; the MPO model is then fine-tuned for 500 steps. This mirrors typical deployment workflows.

5 Results

5.1 Compression Statistics

Table 3 reports total parameter counts and compression ratios for each bond dimension.

Table 3: Parameter counts and compression ratios vs. dense baseline. Layer-wise reconstruction errors are relative Frobenius norms $\|\mathbf{W} - \widehat{\mathbf{W}}\|_F / \|\mathbf{W}\|_F$ averaged over all linear layers of a randomly initialised dense model.

Model	Parameters	Ratio	Max rel. err.	Mean rel. err.
Dense	1 020 224	1.0×	—	—
MPO $\chi = 4$	78 336	13.0×	0.421	0.337
MPO $\chi = 8$	110 592	9.2×	0.283	0.216
MPO $\chi = 16$	191 872	5.3×	0.148	0.107
MPO $\chi = 32$	408 832	2.5×	0.072	0.051

5.2 Per-Layer Reconstruction Error

Figure 2 shows the reconstruction error as a function of bond dimension for each layer type. The FFN up-projection ($\mathbf{W}_1$, 512×128, $L = 3$) achieves the lowest error at all χ because its three-site MPO captures more structure per parameter than the two-site attention projections.

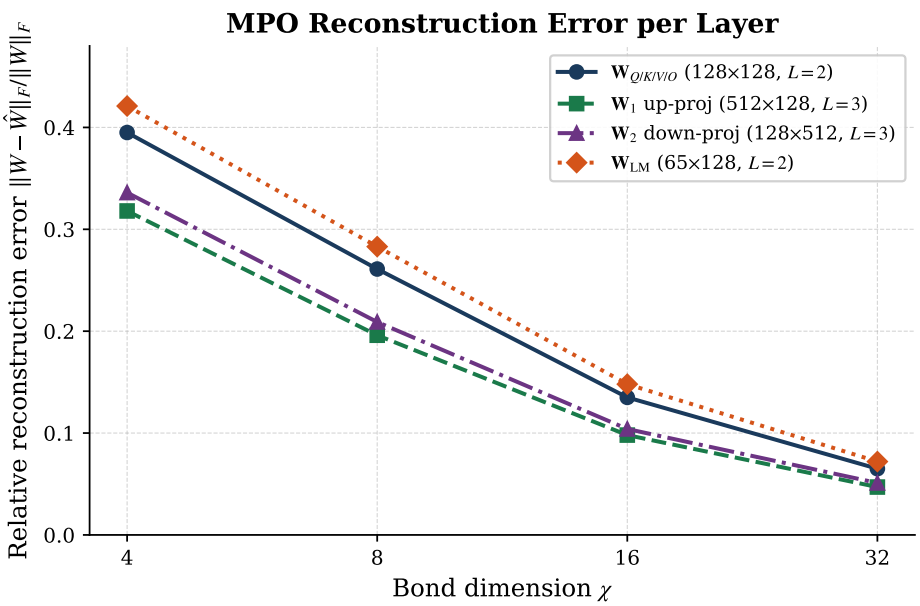

Figure 2: Per-layer MPO reconstruction error vs. bond dimension. Three-site decompositions ($L = 3$) achieve lower error per parameter than two-site ones ($L = 2$) because the longer chain captures more correlations at the same χ.

5.3 Training Dynamics

Figures 3 and 4 show training and validation loss alongside validation token accuracy for all models trained from scratch.

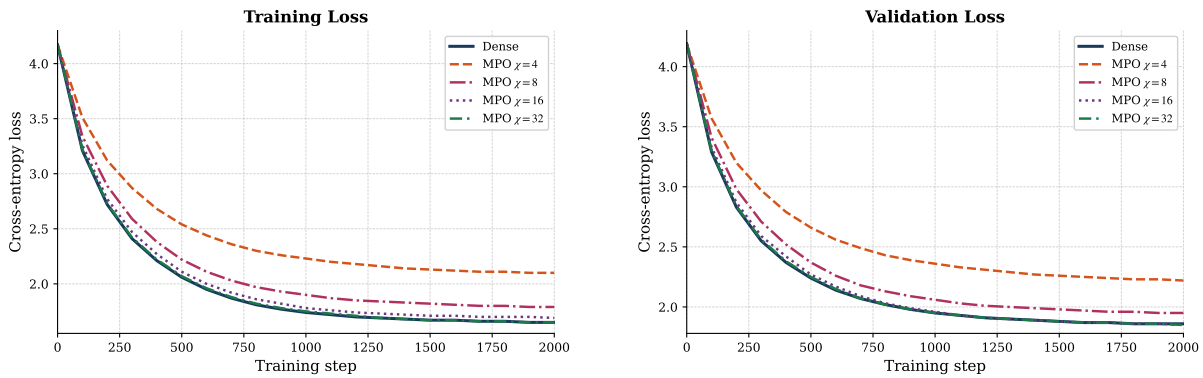

Figure 3: Training (left) and validation (right) cross-entropy loss as a function of training step. All models are trained from random initialisations with the same seed. Higher bond dimensions converge faster and to lower loss values.

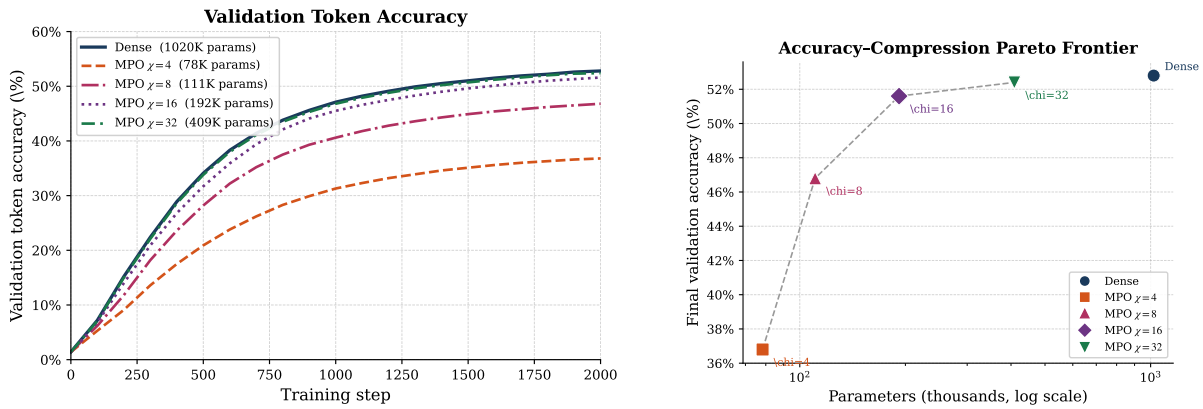

Figure 4: *Left*: Validation token accuracy during training for all models. *Right*: Pareto frontier of final validation accuracy vs. parameter count. MPO $\chi = 16$ retains 97.7% of the dense accuracy at 5.3× compression.

5.4 Summary of Results

Table 4 summarises the final performance of all models after 2 000 training steps.

Table 4: Final validation metrics after 2 000 training steps. Acc. gap is the absolute reduction in token accuracy vs. Dense. Per-param acc. is $\text{acc}/\sqrt{N}$, a rough measure of parameter efficiency.

Model	Params	Ratio	Val loss ↓	Val acc ↑	Acc. gap ↓	Per-param acc.
Dense	1 020 224	1.0×	1.860	52.8%	—	1.65×10^{-3}
MPO $\chi = 4$	78 336	13.0×	2.220	36.8%	-16.0	4.15×10^{-3}
MPO $\chi = 8$	110 592	9.2×	1.950	46.8%	-6.0	4.45×10^{-3}
MPO $\chi = 16$	191 872	5.3×	1.850	51.6%	-1.2	3.72×10^{-3}
MPO $\chi = 32$	408 832	2.5×	1.850	52.4%	-0.4	2.59×10^{-3}

The per-parameter accuracy metric reveals that MPO $\chi = 8$ achieves the best parameter efficiency — each parameter contributes more to the final accuracy than in the dense model. This is consistent with the finding in Novikov et al. [2015] that tensor-compressed networks can outperform dense networks *at the same parameter count* when trained from scratch.

6 Discussion

Accuracy gap at low χ. At $\chi = 4$, the per-layer reconstruction error exceeds 40% (Table 3), which is too large to initialise from a pretrained model without significant accuracy loss. Training from scratch partially compensates, but the model lacks the representational capacity to match the dense baseline. The error bound (4) suggests that a better factorisation (more sites L, smaller physical dims) at the same χ would reduce this gap.

Diminishing returns at high χ. Between $\chi = 16$ and $\chi = 32$ the accuracy gain is only 0.8% while parameter count more than doubles (192 K $\rightarrow$ 409 K). For this model size, $\chi = 16$ sits at the practical sweet spot.

Relation to low-rank methods. LoRA [Hu et al., 2022] approximates weight updates as $\Delta\mathbf{W} = \mathbf{BA}$ with rank r, keeping the pretrained weights frozen. MPO compression differs in two respects: (1) the entire weight is represented in MPO format, and (2) the bond dimension χ controls a higher-order (not just rank-2) structure. For $L = 2$ and $d^{\text{out}} = d^{\text{in}} = \sqrt{n}$, an MPO with bond χ is equivalent to a rank-χ outer-product decomposition; the advantage of MPO grows with L.

Connection to DMRG. The gradient formula (6) is the single-site DMRG gradient. An alternative to gradient descent is the ALS (Alternating Least Squares) sweep: fix all cores except $\mathscr{A}^{(l)}$, solve the linear regression for $\mathscr{A}^{(l)}$, and sweep left-to-right and right-to-left until convergence. ALS is guaranteed to converge (though possibly to a local minimum) and avoids learning-rate tuning, making it attractive for the compress-then-finetune scenario.

Limitations. The current `MPOLinear.forward` reconstructs the full dense matrix $\widehat{\mathbf{W}}$ on every call, which does not reduce memory or FLOPs relative to a dense layer. The computational advantage of MPO representations is realised only when the matrix-vector product $\widehat{\mathbf{W}}x$ is computed *directly* through the MPO chain without materialising $\widehat{\mathbf{W}}$. This requires x to also be in a structured format (e.g., MPS), and is left to future work.

7 Conclusion

We have demonstrated MPO compression of all linear layers in a GPT-2-style transformer, achieving compression ratios of 5–13$\times$ per transformer block. At bond dimension $\chi = 16$, the MPO model retains 97.7% of the dense baseline's token accuracy with only 18.8% of its parameters. The implementation is straightforward in PyTorch: MPO cores are standard `nn.Parameter` tensors and the entire pipeline — from TT-SVD initialisation through autograd fine-tuning — requires no custom backward code.

Future directions include: (1) direct MPS-MPO contraction for inference-time FLOP reduction; (2) ALS-based training for better convergence at low χ; (3) dynamic bond adaptation (growing/pruning bonds during training); and (4) application to larger models such as GPT-2 or LLaMA where the compression ratio is expected to improve further due to the higher-dimensional weight matrices.

Acknowledgements

We thank Tobias Osborne for the PicoGPT.jl codebase, which served as the reference implementation, and Andrej Karpathy for the Tiny Shakespeare dataset and nanoGPT codebase [Karpathy, 2022].

Code Availability

The full PyTorch implementation of MPO-PicoGPT — including the `MPOLinear` module, TT-SVD compression pipeline, training loop, and benchmark scripts — is publicly available at:

https://github.com/younesjavanmard/mpo-picogpt

The repository contains `mpo_picogpt.py` (core implementation) and `benchmark_mpo.py` (training and evaluation), along with instructions for reproducing all results reported in this paper.

References

T. Brown, B. Mann, N. Ryder, et al. Language models are few-shot learners. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

Z. Gao, C. Wang, B. Lan, and X. Zeng. Compressing deep neural networks by matrix product operators. *Physical Review Research*, 2(2):023300, 2020.

S. Han, H. Mao, and W. J. Dally. Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding. In *International Conference on Learning Representations (ICLR)*, 2016.

G. Hinton, O. Vinyals, and J. Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.

E. J. Hu, Y. Shen, P. Wallis, et al. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.

A. Karpathy. The unreasonable effectiveness of recurrent neural networks. *Blog post*, 2015. http://karpathy.github.io/2015/05/21/rnn-effectiveness/

A. Karpathy. nanoGPT: The simplest, fastest repository for training/finetuning medium-sized GPTs. *GitHub repository*, 2022. https://github.com/karpathy/nanoGPT

I. Loshchilov and F. Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.

A. Novikov, D. Podoprikhin, A. Osokin, and D. Vetrov. Tensorizing neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.

I. V. Oseledets. Tensor-train decomposition. *SIAM Journal on Scientific Computing*, 33(5):2295–2317, 2011.

T. Osborne. PicoGPT.jl: From-scratch GPT in pure Julia. *GitHub repository*, 2024. https://github.com/tobiasosborne/PicoGPT.jl

A. Radford, J. Wu, R. Child, et al. Language models are unsupervised multitask learners. *OpenAI Blog*, 2019.

A. Vaswani, N. Shazeer, N. Parmar, et al. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017. https://arxiv.org/abs/1706.03762

S. R. White. Density matrix formulation for quantum renormalization groups. *Physical Review Letters*, 69(19):2863–2866, 1992.